



COVID-19 Exploratory Data Analysis

Dataset: covid_19_clean_complete.csv

Done by: Sulav Pandey

Roll No.: 024BSCIT043

Section: 'B'

A time-series exploration of global COVID-19 case data — confirmed cases, deaths, recoveries, and active cases across countries and dates — covering data cleaning, statistical summaries, outlier detection, and correlation analysis.

```
In [40]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
import seaborn as sns
```

```
In [41]: df = pd.read_csv('covid_19_clean_complete.csv')
df.head(5)
```

```
Out[41]:
```

	Province/State	Country/Region	Lat	Long	Date	Confirmed	Deaths	Recov
0	NaN	Afghanistan	33.93911	67.709953	2020-01-22	0	0	
1	NaN	Albania	41.15330	20.168300	2020-01-22	0	0	
2	NaN	Algeria	28.03390	1.659600	2020-01-22	0	0	
3	NaN	Andorra	42.50630	1.521800	2020-01-22	0	0	
4	NaN	Angola	-11.20270	17.873900	2020-01-22	0	0	

```
In [42]: df.tail(5)
```

Out[42]:

	Province/State	Country/Region	Lat	Long	Date	Confirmed	Deaths
49063	NaN	Sao Tome and Principe	0.186400	6.613100	2020-07-27	865	14
49064	NaN	Yemen	15.552727	48.516388	2020-07-27	1691	483
49065	NaN	Comoros	-11.645500	43.333300	2020-07-27	354	7
49066	NaN	Tajikistan	38.861000	71.276100	2020-07-27	7235	60
49067	NaN	Lesotho	-29.610000	28.233600	2020-07-27	505	12

In [43]: `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 49068 entries, 0 to 49067
Data columns (total 10 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   Province/State  14664 non-null  object
1   Country/Region  49068 non-null  object
2   Lat             49068 non-null  float64
3   Long            49068 non-null  float64
4   Date            49068 non-null  object
5   Confirmed       49068 non-null  int64
6   Deaths         49068 non-null  int64
7   Recovered       49068 non-null  int64
8   Active          49068 non-null  int64
9   WHO Region      49068 non-null  object
dtypes: float64(2), int64(4), object(4)
memory usage: 3.7+ MB
```

In [44]: `df.describe()`

Out[44]:

	Lat	Long	Confirmed	Deaths	Recovered	Act
count	49068.000000	49068.000000	4.906800e+04	49068.000000	4.906800e+04	4.906800e+04
mean	21.433730	23.528236	1.688490e+04	884.179160	7.915713e+03	8.085012e+03
std	24.950320	70.442740	1.273002e+05	6313.584411	5.480092e+04	7.625890e+04
min	-51.796300	-135.000000	0.000000e+00	0.000000	0.000000e+00	-1.400000e+00
25%	7.873054	-15.310100	4.000000e+00	0.000000	0.000000e+00	0.000000e+00
50%	23.634500	21.745300	1.680000e+02	2.000000	2.900000e+01	2.600000e+01
75%	41.204380	80.771797	1.518250e+03	30.000000	6.660000e+02	6.060000e+02
max	71.706900	178.065000	4.290259e+06	148011.000000	1.846641e+06	2.816444e+06

Checking the types of data:

In [45]: `df.dtypes`

```
Out[45]: Province/State    object
Country/Region    object
Lat                float64
Long               float64
Date               object
Confirmed          int64
Deaths            int64
Recovered          int64
Active            int64
WHO Region        object
dtype: object
```

Note: `Date` is currently an `object` (string), not a real date. For a time-series dataset this matters a lot — we can't sort, plot over time, or extract month/week from a string properly.

```
In [46]: # Convert Date column to actual datetime type
df['Date'] = pd.to_datetime(df['Date'])
df.dtypes
```

```
Out[46]: Province/State    object
Country/Region    object
Lat                float64
Long               float64
Date              datetime64[ns]
Confirmed          int64
Deaths            int64
Recovered          int64
Active            int64
WHO Region        object
dtype: object
```

```
In [47]: #Dropping irrelevant columns
df = df.drop(['WHO Region'], axis=1)
df.head(5)
```

```
Out[47]:
```

	Province/State	Country/Region	Lat	Long	Date	Confirmed	Deaths	Recov
0	NaN	Afghanistan	33.93911	67.709953	2020-01-22	0	0	
1	NaN	Albania	41.15330	20.168300	2020-01-22	0	0	
2	NaN	Algeria	28.03390	1.659600	2020-01-22	0	0	
3	NaN	Andorra	42.50630	1.521800	2020-01-22	0	0	
4	NaN	Angola	-11.20270	17.873900	2020-01-22	0	0	

```
In [48]: #Renaming the columns
df = df.rename(columns={"Country/Region": "Country", "Province/State": "Province"})
df.head(5)
```

```
Out[48]:
```

	Province	Country	Lat	Long	Date	Confirmed	Deaths	Recovered	Acti
0	NaN	Afghanistan	33.93911	67.709953	2020-01-22	0	0	0	
1	NaN	Albania	41.15330	20.168300	2020-01-22	0	0	0	
2	NaN	Algeria	28.03390	1.659600	2020-01-22	0	0	0	
3	NaN	Andorra	42.50630	1.521800	2020-01-22	0	0	0	
4	NaN	Angola	-11.20270	17.873900	2020-01-22	0	0	0	

```
In [49]: #Dropping the duplicate rows
df.shape
```

```
Out[49]: (49068, 9)
```

```
In [50]: duplicate_rows_df = df[df.duplicated()]
print("number of duplicate rows: ", duplicate_rows_df.shape)
```

```
number of duplicate rows: (0, 9)
```

```
In [51]: df.count()      # Used to count the number of rows
```

```
Out[51]: Province      14664
Country      49068
Lat          49068
Long         49068
Date         49068
Confirmed    49068
Deaths       49068
Recovered    49068
Active       49068
dtype: int64
```

```
In [52]: # Take only the most recent date so each country contributes one row
latest_date = df['Date'].max()
latest_df = df[df['Date'] == latest_date].copy()
print(f"Snapshot date: {latest_date}")
latest_df.shape
```

Snapshot date: 2020-07-27 00:00:00

```
Out[52]: (261, 9)
```

```
In [53]: df.count()
```

```
Out[53]: Province      14664
Country      49068
Lat          49068
Long         49068
Date         49068
Confirmed    49068
Deaths       49068
Recovered    49068
Active       49068
dtype: int64
```

```
In [54]: #Dropping the missing or null values.
print(df.isnull().sum())
```

```
Province      34404
Country        0
Lat           0
Long           0
Date           0
Confirmed      0
Deaths         0
Recovered      0
Active         0
dtype: int64
```

```
In [55]: # Instead of dropna() (which would wipe out most rows), fill missing
# Province values with a placeholder since the country-level data is still valid
df['Province'] = df['Province'].fillna('Not Reported')

print(df.isnull().sum()) # Province should now show 0 missing
```

```
Province      0
Country       0
Lat           0
Long          0
Date          0
Confirmed     0
Deaths        0
Recovered     0
Active        0
dtype: int64
```

In [56]: `df.head()`

Out[56]:

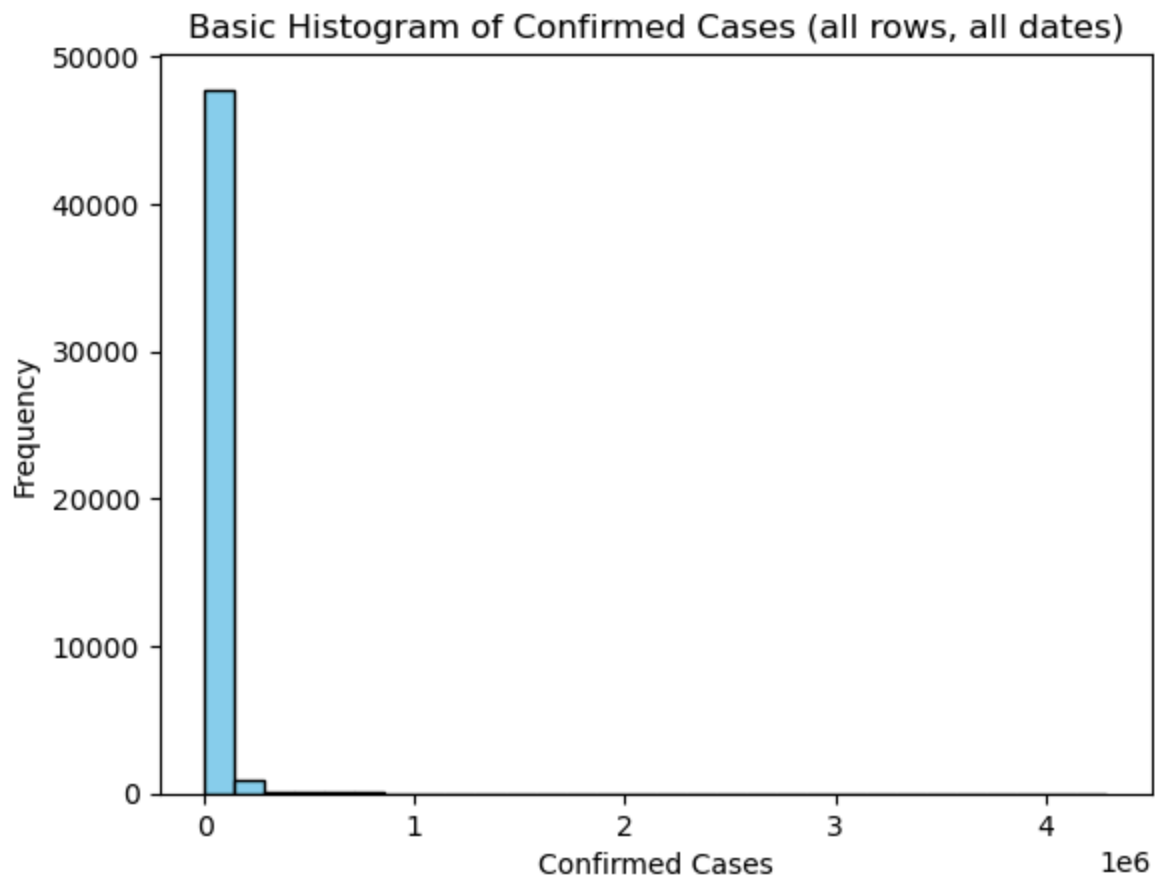
	Province	Country	Lat	Long	Date	Confirmed	Deaths	Recovered	Active
0	Not Reported	Afghanistan	33.93911	67.709953	2020-01-22	0	0	0	
1	Not Reported	Albania	41.15330	20.168300	2020-01-22	0	0	0	
2	Not Reported	Algeria	28.03390	1.659600	2020-01-22	0	0	0	
3	Not Reported	Andorra	42.50630	1.521800	2020-01-22	0	0	0	
4	Not Reported	Angola	-11.20270	17.873900	2020-01-22	0	0	0	

Note on the histogram below: `Confirmed` is a *cumulative* running total per row (it only grows day by day for each country). A plain histogram of every row mixes early-pandemic numbers (small) with later numbers (huge) — it will look heavily skewed. We plot it anyway since the sample asks for it, but the skew itself is a useful observation, not a bug.

```
In [57]: # Plotting a basic histogram
plt.hist(x=df['Confirmed'], bins=30, color='skyblue', edgecolor='black')

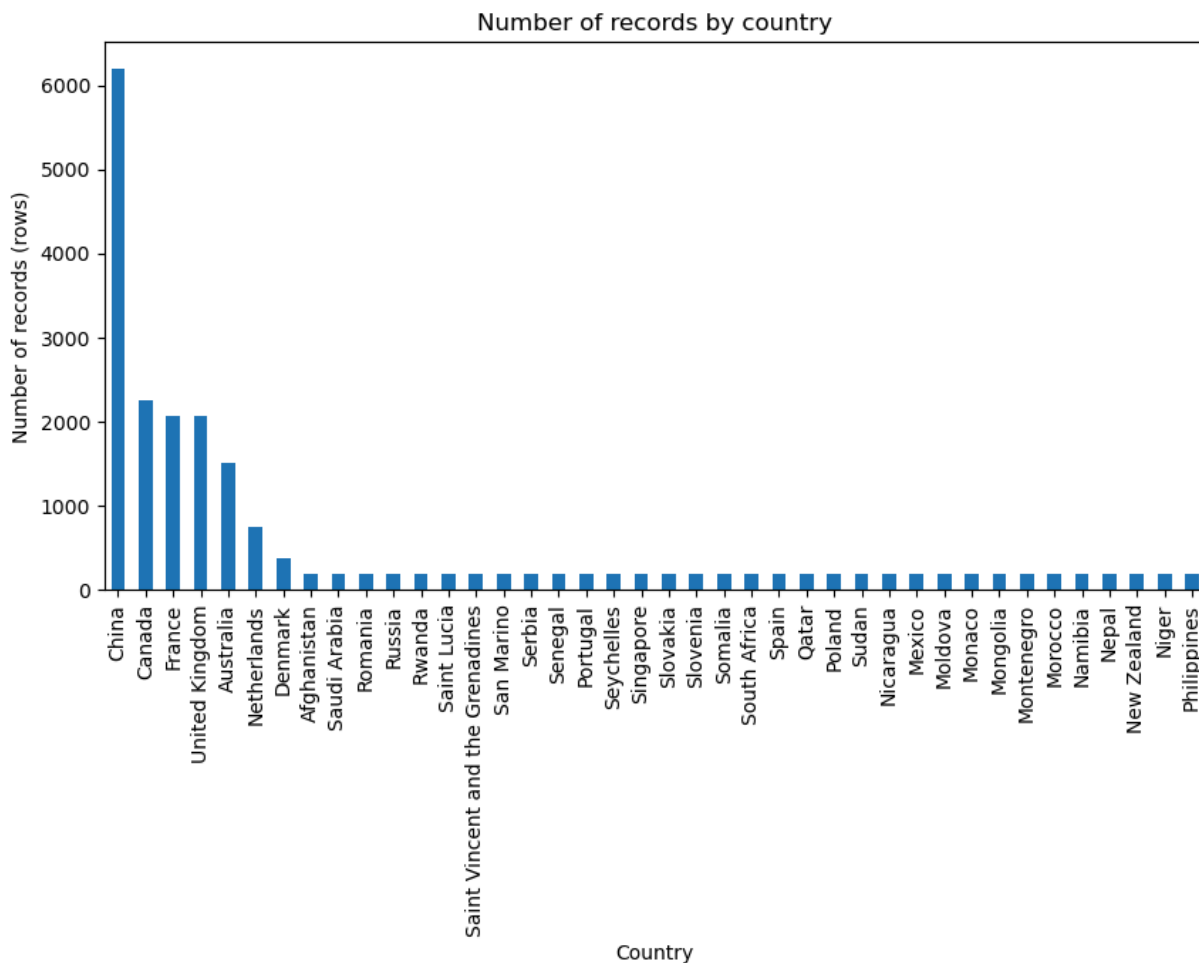
# Adding labels and title
plt.xlabel('Confirmed Cases')
plt.ylabel('Frequency')
plt.title('Basic Histogram of Confirmed Cases (all rows, all dates)')

# Display the plot
plt.show()
```



```
In [58]: #Plot different features against one another (scatter), against frequency (histogram)
df.Country.value_counts().nlargest(40).plot(kind='bar', figsize=(10,5))
plt.title("Number of records by country")
plt.ylabel('Number of records (rows)')
plt.xlabel('Country')
```

```
Out[58]: Text(0.5, 0, 'Country')
```

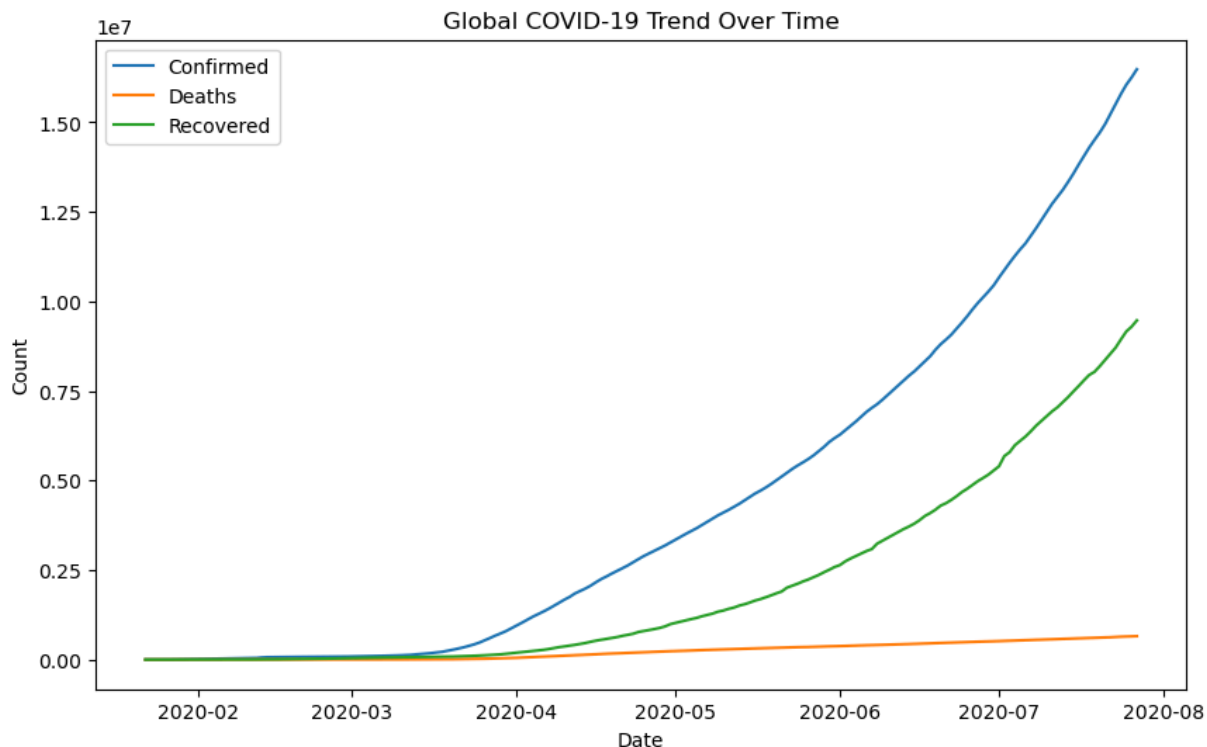


Notice every country has roughly the *same* number of records — that's expected, since each country gets one row per date over the same date range. This bar chart is really confirming the dataset's structure rather than showing variation.

Extra step: Global trend over time

```
In [59]: # Aggregate worldwide totals for each date
global_trend = df.groupby('Date')[['Confirmed', 'Deaths', 'Recovered']].sum()

plt.figure(figsize=(10,6))
plt.plot(global_trend.index, global_trend['Confirmed'], label='Confirmed')
plt.plot(global_trend.index, global_trend['Deaths'], label='Deaths')
plt.plot(global_trend.index, global_trend['Recovered'], label='Recovered')
plt.xlabel('Date')
plt.ylabel('Count')
plt.title('Global COVID-19 Trend Over Time')
plt.legend()
plt.show()
```

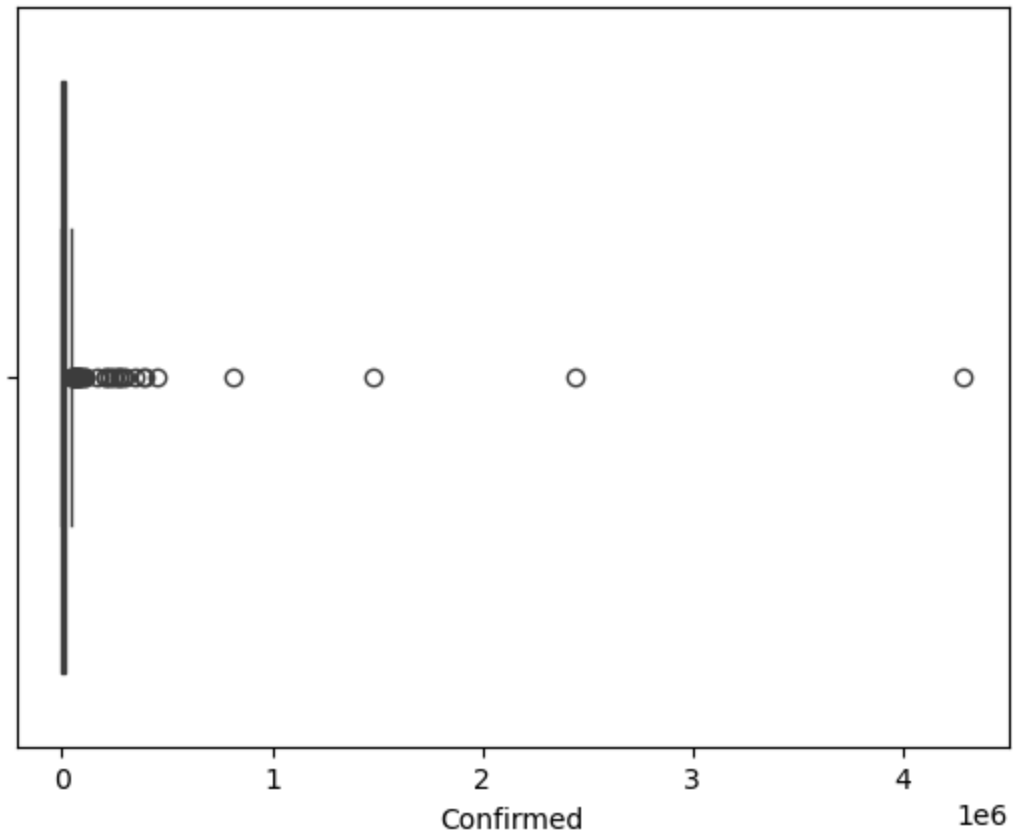



Detecting Outliers

Adapted step: Running a boxplot on `Confirmed` across *all* rows (all dates) would mostly show the early-pandemic near-zero values as a dense cluster and later huge values as "outliers" — which isn't really what outlier detection is meant to find here. Instead, we take a **snapshot of the latest date only**, so each country contributes exactly one value, and outliers genuinely mean "countries with unusually high case counts compared to the rest".

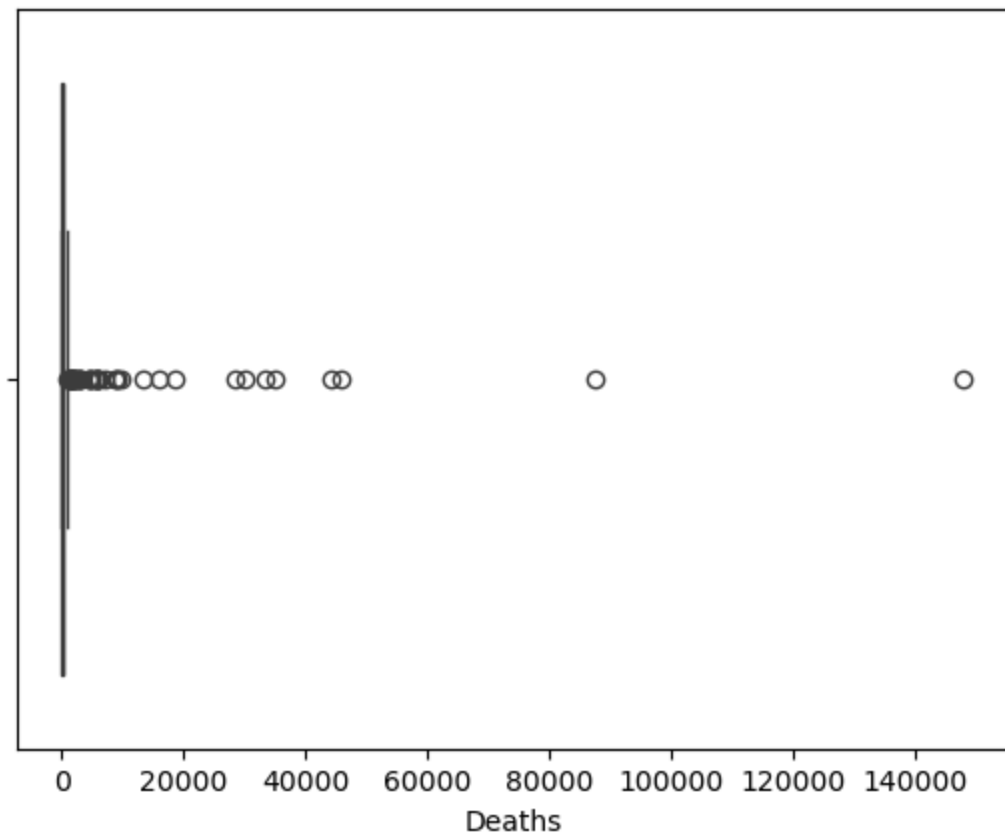
```
In [60]: #Detecting Outliers
sns.boxplot(x=latest_df['Confirmed'])
```

```
Out[60]: <Axes: xlabel='Confirmed'>
```



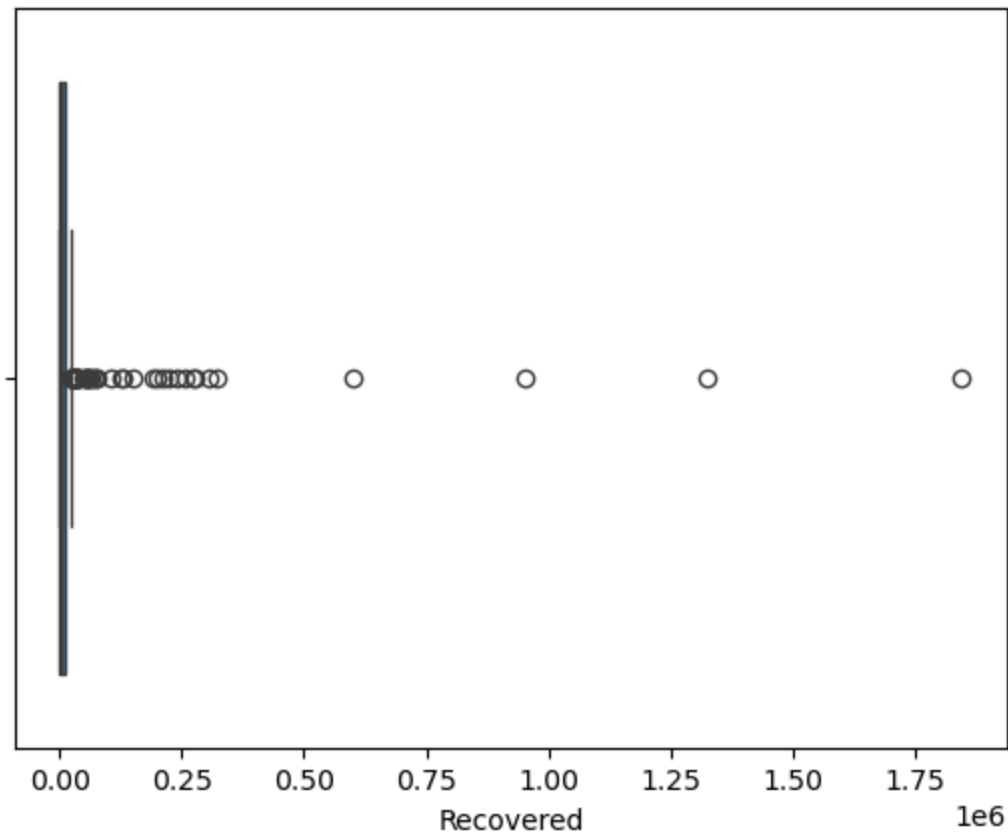
```
In [61]: sns.boxplot(x=latest_df['Deaths'])
```

```
Out[61]: <Axes: xlabel='Deaths'>
```



```
In [62]: sns.boxplot(x=latest_df['Recovered'])
```

```
Out[62]: <Axes: xlabel='Recovered'>
```



The countries appearing as individual dots far to the right of the box are the outliers — these are countries with case counts far above the typical (median) country, e.g. the US, India, Brazil. This matches what we'd expect from real-world COVID spread, which is a good sanity check that the data is clean.

For Correlation Matrix just use numerical value column as selected below

```
In [63]: selected_columns = ['Confirmed', 'Deaths', 'Recovered', 'Active']
df_selected = latest_df[selected_columns]

plt.figure(figsize=(15, 10))

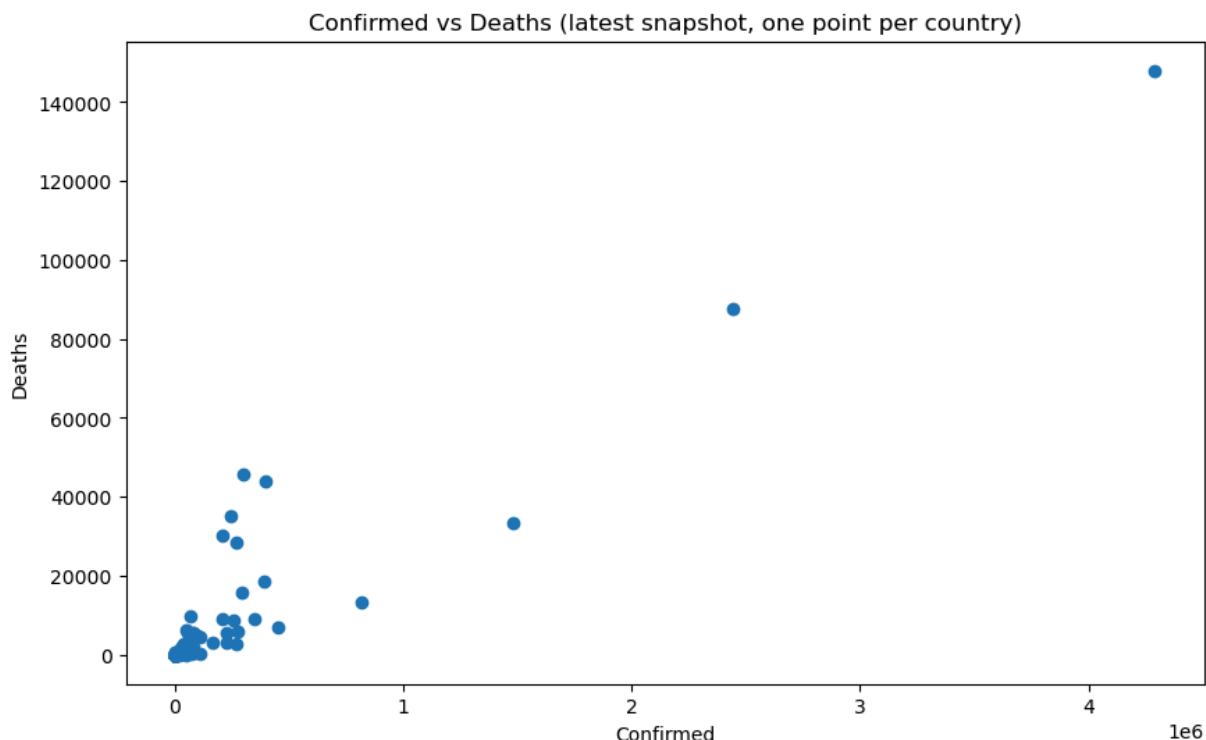
# Using Seaborn to create a heatmap
sns.heatmap(df_selected.corr(), annot=True, fmt='.2f', cmap='Pastel2', linewidths=2)

plt.title('Correlation Heatmap (latest snapshot)')
plt.show()
```



Note: We use `latest_df` (one row per country) here instead of the full `df` (all dates). Correlating cumulative totals across all dates would be misleading — Confirmed and Deaths would look almost perfectly correlated simply because both grow over time for every country, regardless of any real relationship between them. Using one snapshot avoids that trap.

```
In [64]: #scatter plot
fig, ax = plt.subplots(figsize=(10,6))
ax.scatter(latest_df['Confirmed'], latest_df['Deaths'])
ax.set_xlabel('Confirmed')
ax.set_ylabel('Deaths')
ax.set_title('Confirmed vs Deaths (latest snapshot, one point per country)')
plt.show()
```



Statistical Summary

`describe()` already gives mean/std/min/max, but doesn't put mean and median side-by-side, which is the comparison that actually tells you something about skew.

```
In [65]: # Statistical Summary
stats = latest_df[["Confirmed", "Deaths", "Recovered", "Active"]].agg(
    ["mean", "median", "std", "min", "max"]
)

print(stats)
```

	Confirmed	Deaths	Recovered	Active
mean	6.314362e+04	2505.885057	3.627620e+04	2.436154e+04
median	1.879000e+03	42.000000	9.750000e+02	4.490000e+02
std	3.265442e+05	12020.969891	1.624353e+05	1.810031e+05
min	1.000000e+00	0.000000	0.000000e+00	0.000000e+00
max	4.290259e+06	148011.000000	1.846641e+06	2.816444e+06

How to read this:

- The **mean** is the average — but it gets pulled upward by a few huge countries (US, India, Brazil).
- The **median** is the "typical" country, unaffected by those extremes.
- If mean is much bigger than median → the data is **right-skewed**, meaning most countries have low case counts and a handful have very high ones. We confirm this properly in the next section.
- A large **std** relative to the mean means countries vary wildly — which is expected here, since a tiny island nation and the US are in the same column.

Skewness and Kurtosis

These two numbers describe the *shape* of the distribution instead of just its center.

- **Skewness** tells you which way the data leans.
- **Kurtosis** tells you how extreme the outliers are (how "heavy-tailed" the distribution is).

```
In [66]: print("Skewness")
print(latest_df[["Confirmed", "Deaths", "Recovered", "Active"]].skew())

print("\nKurtosis")
print(latest_df[["Confirmed", "Deaths", "Recovered", "Active"]].kurt())
```

Skewness

```
Confirmed    10.292396
Deaths       8.832249
Recovered    8.254019
Active      14.382539
dtype: float64
```

Kurtosis

```
Confirmed    120.113742
Deaths       93.169161
Recovered    78.015108
Active      220.216108
dtype: float64
```

Interpretation:

- **Positive skewness** (which you'll see here) means most countries have relatively few cases, while a small number have exceptionally large outbreaks — the long tail is on the high side.
- **High kurtosis** means the distribution has extreme outliers compared to a normal distribution. A few countries pull the tail far to the right, which is exactly what the boxplots showed earlier.

This is the statistical confirmation of what we already saw visually in the histogram and boxplots — now we have numbers backing up that observation instead of just eyeballing the chart.

Coefficient of Variation (CV)

Standard deviation alone is hard to interpret because it depends on the scale of the numbers. CV fixes this by expressing spread as a **percentage of the mean**, so you can compare variability across columns of very different magnitudes (e.g. Deaths vs Confirmed).

```
In [67]: cv = (latest_df[["Confirmed", "Deaths", "Recovered", "Active"]].std() /
              latest_df[["Confirmed", "Deaths", "Recovered", "Active"]].mean()) * 100
```

```
print(cv)
```

```
Confirmed    517.145243
Deaths       479.709548
Recovered    447.773815
Active       742.986890
dtype: float64
```

Interpretation: A higher CV means that column is more dispersed relative to its own average — i.e., countries differ from each other more sharply on that metric. Comparing CV across Confirmed/Deaths/Recovered/Active tells you which of these four varies the *most* between countries, in a way that raw standard deviation can't, since raw std would just say "Confirmed has bigger numbers" without telling you about relative spread.

Quartile Analysis (IQR)

The IQR (Interquartile Range) measures the spread of the **middle 50%** of countries — i.e., it deliberately ignores the extreme top and bottom, which makes it a more honest measure of "typical spread" than the full min-max range, especially with the outliers we already found.

```
In [68]: summary = latest_df[["Confirmed", "Deaths", "Recovered", "Active"]].describe().T
summary["IQR"] = summary["75%"] - summary["25%"]
summary
```

```
Out[68]:
```

	count	mean	std	min	25%	50%	75%	max
Confirmed	261.0	63143.620690	326544.230543	1.0	311.0	1879.0	17975.0	4290259.0
Deaths	261.0	2505.885057	12020.969891	0.0	3.0	42.0	347.0	148011.0
Recovered	261.0	36276.195402	162435.303984	0.0	128.0	975.0	8601.0	1846641.0
Active	261.0	24361.540230	181003.050100	0.0	13.0	449.0	5119.0	2816444.0

Feature Engineering:

Raw counts (Confirmed, Deaths, Recovered) aren't directly comparable across countries of very different population/outbreak sizes. Converting them to **rates** (percentages of Confirmed) makes cross-country comparison meaningful — a small country and a huge country can now be compared fairly on outcome quality rather than raw scale.

Note: this only adds new columns, nothing existing is modified.

```
In [69]: latest_df["Recovery Rate (%)"] = (latest_df["Recovered"] / latest_df["Confirmed"])
latest_df["Death Rate (%)"] = (latest_df["Deaths"] / latest_df["Confirmed"]) * 100
latest_df["Active Rate (%)"] = (latest_df["Active"] / latest_df["Confirmed"]) * 100
```

```
latest_df[["Country", "Recovery Rate (%)", "Death Rate (%)", "Active Rate (%)"]].he
```

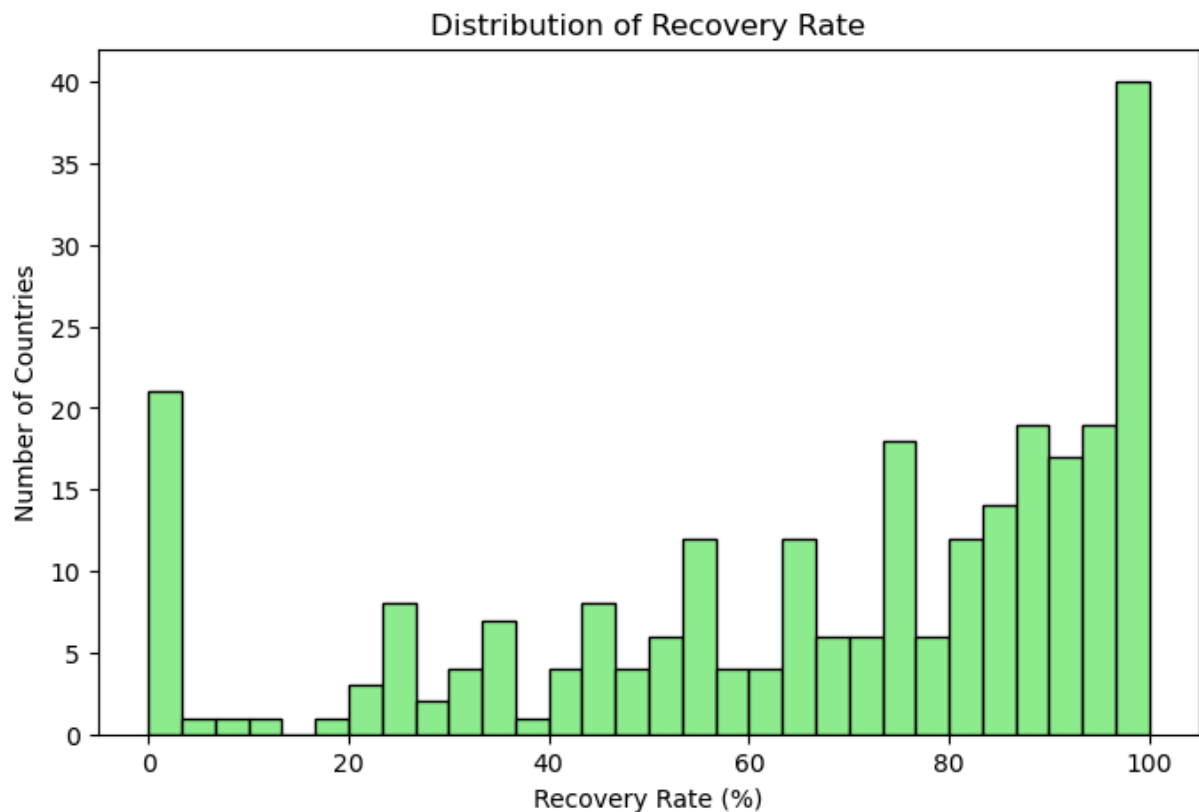
Out[69]:

	Country	Recovery Rate (%)	Death Rate (%)	Active Rate (%)
48807	Afghanistan	69.486805	3.499435	27.013761
48808	Albania	56.250000	2.950820	40.799180
48809	Algeria	67.339935	4.157581	28.502485
48810	Andorra	88.533627	5.733186	5.733186
48811	Angola	25.473684	4.315789	70.210526

```
In [70]: # Fill any resulting NaN values (e.g., where Confirmed is zero)
latest_df[["Recovery Rate (%)", "Death Rate (%)", "Active Rate (%)"]] = (
    latest_df[["Recovery Rate (%)", "Death Rate (%)", "Active Rate (%)"]]
    .fillna(0)
)
```

Distribution of Recovery Rate

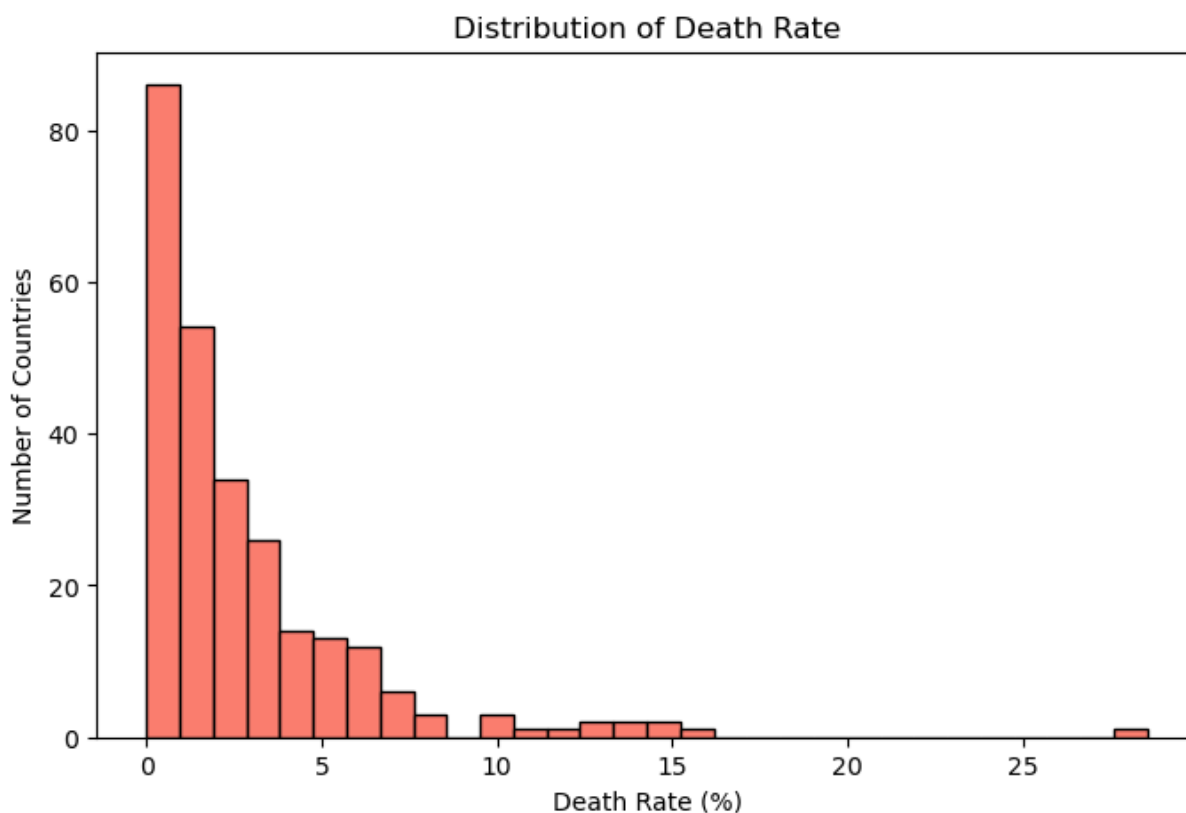
```
In [71]: plt.figure(figsize=(8,5))
plt.hist(latest_df["Recovery Rate (%)"], bins=30, color='lightgreen', edgecolor='b')
plt.title("Distribution of Recovery Rate")
plt.xlabel("Recovery Rate (%)")
plt.ylabel("Number of Countries")
plt.show()
```



Interpretation: Shows whether most countries cluster around a high recovery percentage or whether recovery rate varies a lot. A tight cluster near 100% means most countries recovered nearly everyone who got confirmed; a spread-out histogram means recovery outcomes varied a lot by country — likely reflecting differences in healthcare capacity and how thoroughly each country tracked recoveries.

Distribution of Death Rate

```
In [72]: plt.figure(figsize=(8,5))
plt.hist(latest_df["Death Rate (%)"], bins=30, color='salmon', edgecolor='black')
plt.title("Distribution of Death Rate")
plt.xlabel("Death Rate (%)")
plt.ylabel("Number of Countries")
plt.show()
```

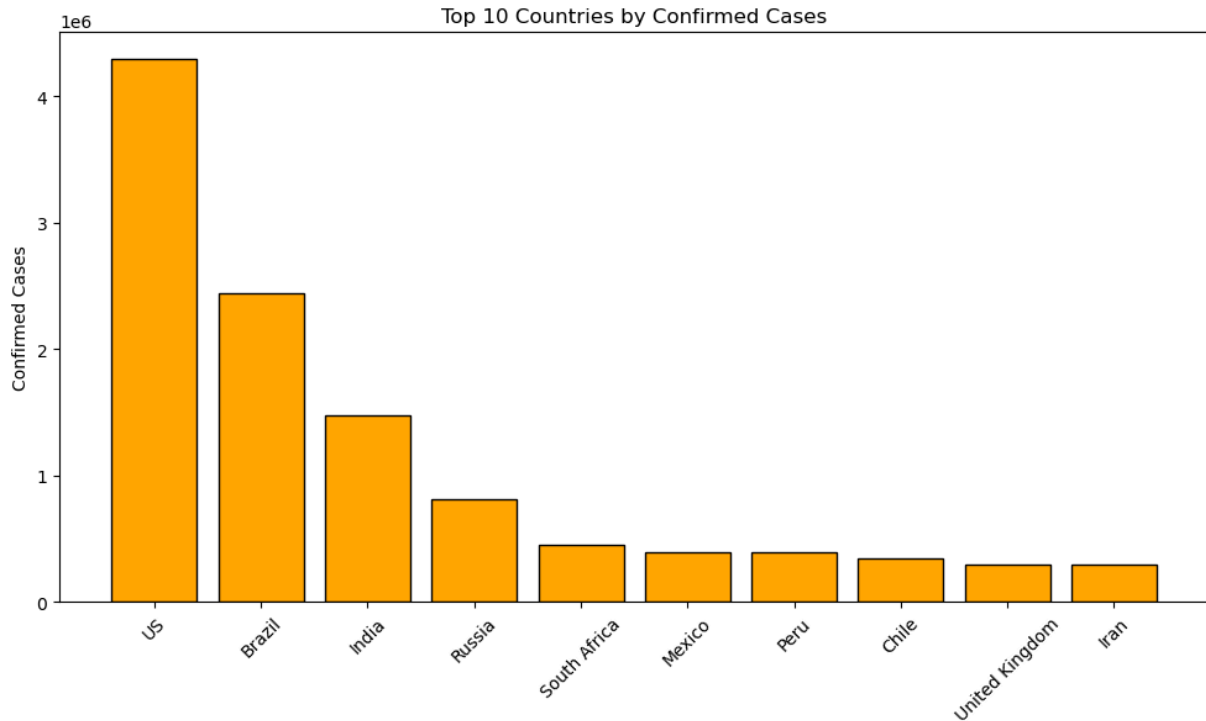


Top 10 Countries by Confirmed Cases (with derived data)

```
In [73]: top10_confirmed = latest_df.nlargest(10, "Confirmed")

plt.figure(figsize=(12,6))
plt.bar(top10_confirmed["Country"], top10_confirmed["Confirmed"], color='orange', e
plt.xticks(rotation=45)
plt.title("Top 10 Countries by Confirmed Cases")
```

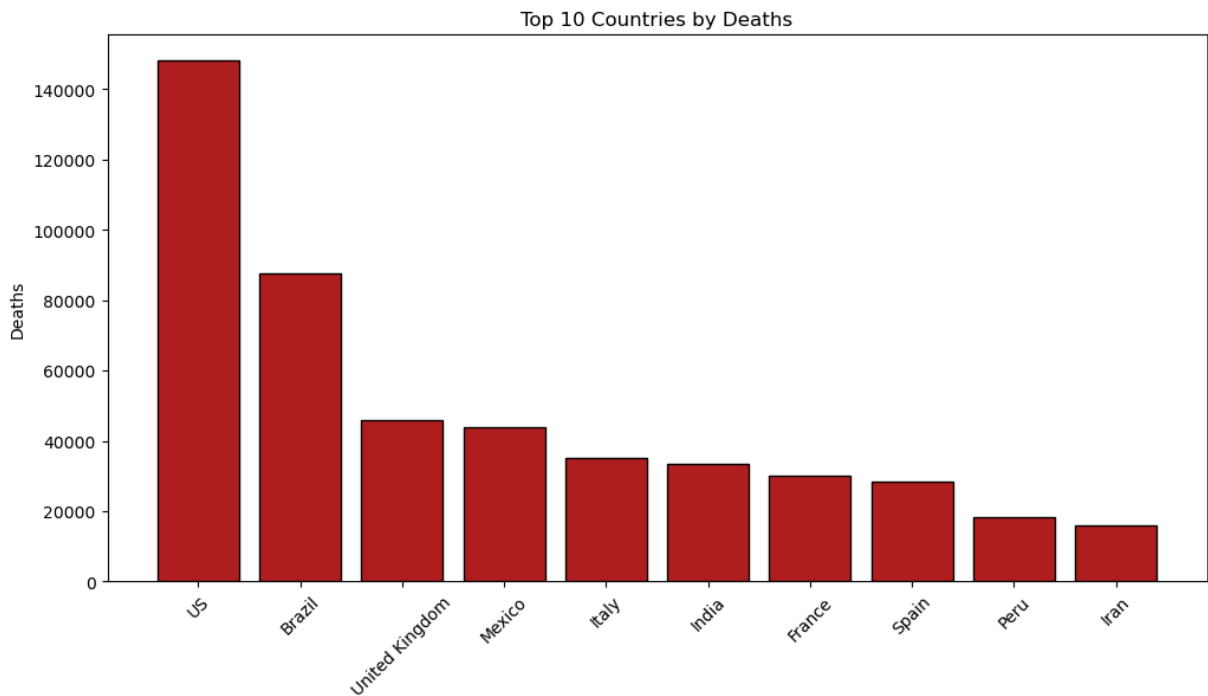
```
plt.ylabel("Confirmed Cases")  
plt.show()
```



Interpretation: Identifies the countries contributing the largest share of confirmed cases worldwide — these are the countries that dominate the global totals we saw in the trend line plot earlier.

Top 10 Countries by Deaths

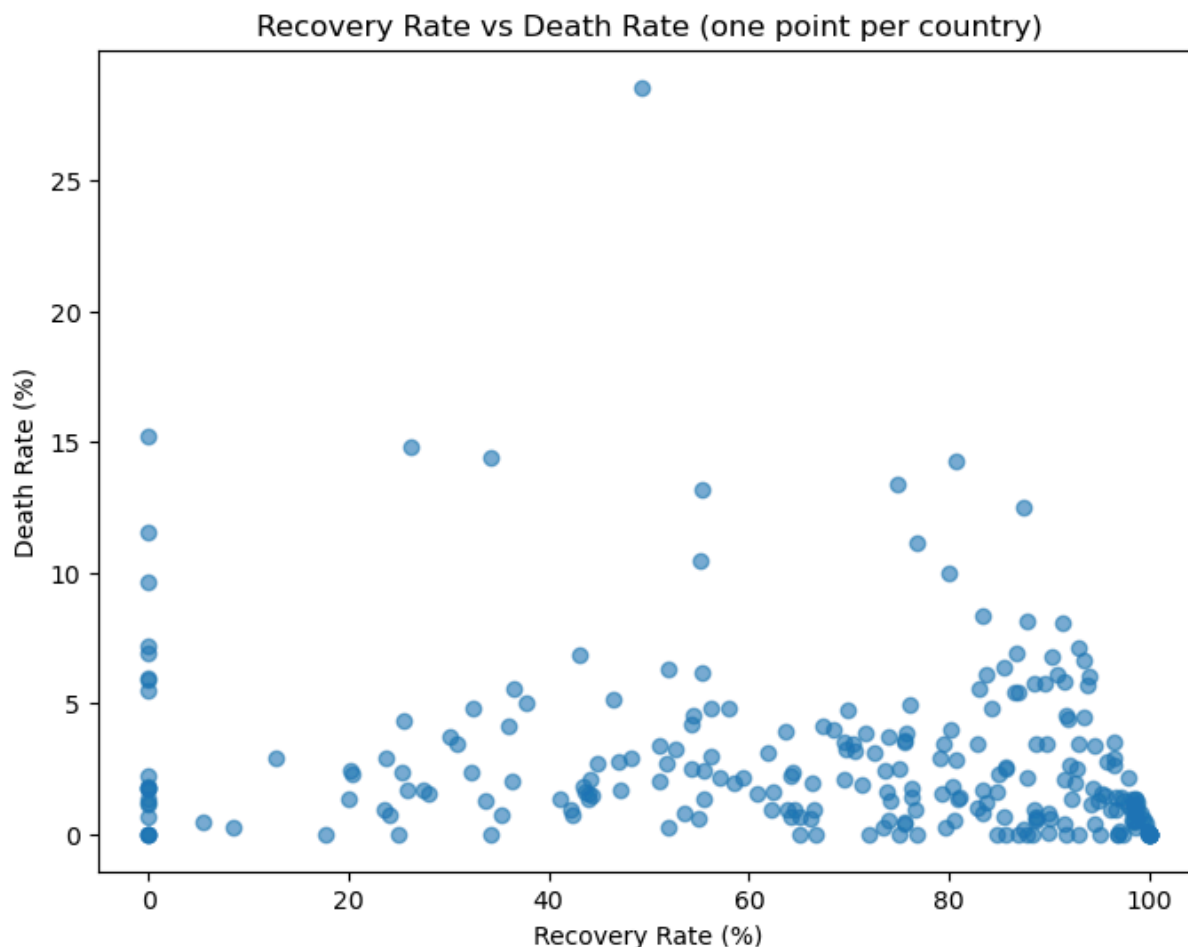
```
In [74]: top10_deaths = latest_df.nlargest(10, "Deaths")  
  
plt.figure(figsize=(12,6))  
plt.bar(top10_deaths["Country"], top10_deaths["Deaths"], color='firebrick', edgecol  
plt.xticks(rotation=45)  
plt.title("Top 10 Countries by Deaths")  
plt.ylabel("Deaths")  
plt.show()
```



Note: Compare this list to the Top 10 by Confirmed above — they won't be identical. A country can have very high confirmed cases but a lower death ranking (better healthcare/younger population), or vice versa. That mismatch is itself an insight worth pointing out in your report.

Recovery Rate vs Death Rate

```
In [75]: plt.figure(figsize=(8,6))
plt.scatter/latest_df["Recovery Rate (%)"], latest_df["Death Rate (%)", alpha=0.6)
plt.xlabel("Recovery Rate (%)")
plt.ylabel("Death Rate (%)")
plt.title("Recovery Rate vs Death Rate (one point per country)")
plt.show()
```



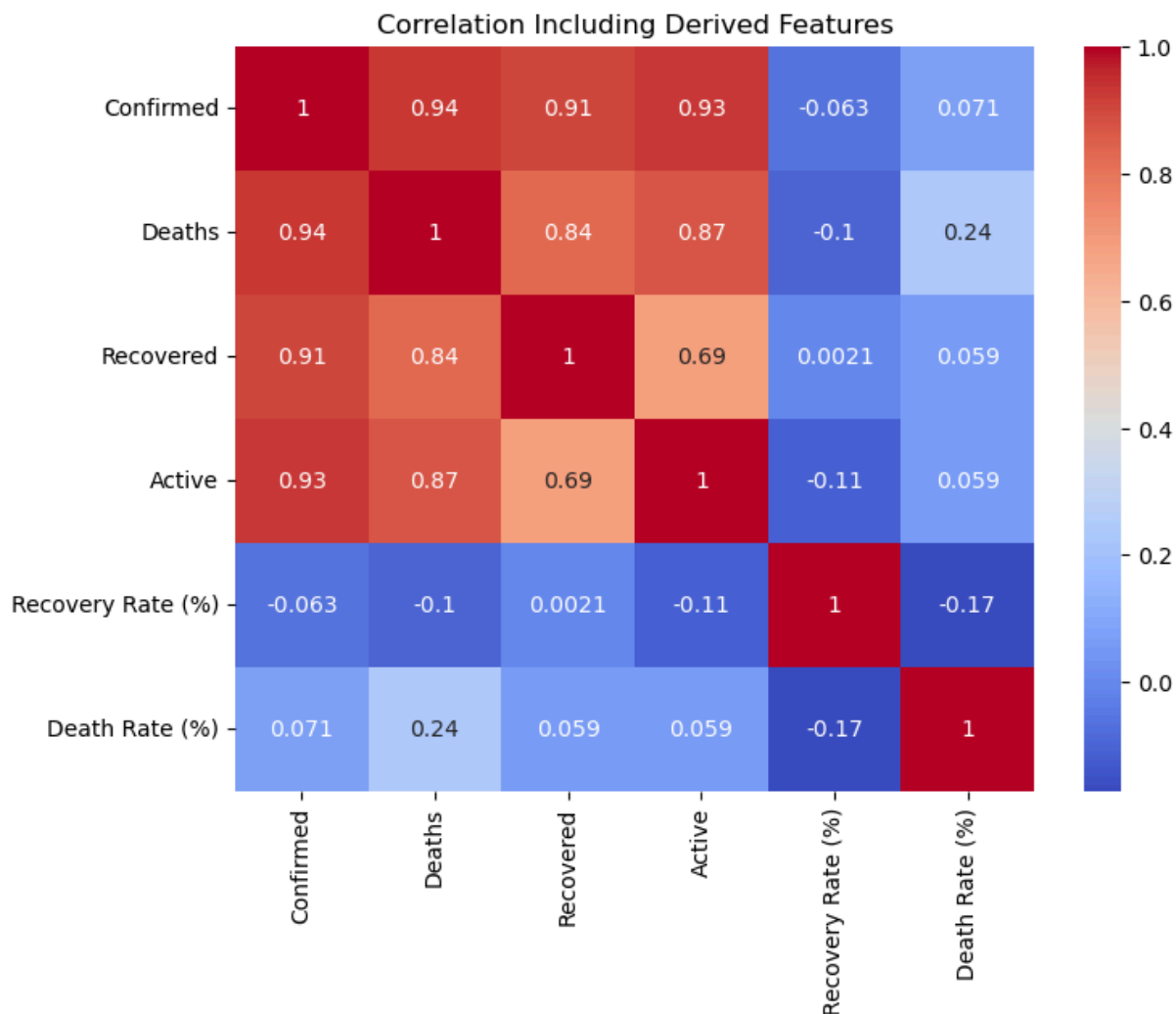
Interpretation: Examines whether countries with higher recovery rates also tended to have lower death rates. If you see a downward trend (top-left to bottom-right), it suggests the two are inversely related, which would make epidemiological sense — better outcomes show up as both higher recovery and lower mortality together.

Correlation Heatmap

Your existing correlation heatmap (Part 1) used only the raw counts. This version adds the rate columns, to see if the *derived* metrics correlate differently than the raw ones did.

```
In [76]: corr_extended = latest_df[
    ["Confirmed", "Deaths", "Recovered", "Active",
     "Recovery Rate (%)", "Death Rate (%)"]
].corr()

plt.figure(figsize=(8,6))
sns.heatmap(corr_extended, annot=True, cmap="coolwarm")
plt.title("Correlation Including Derived Features")
plt.show()
```



Why this matters: Confirmed/Deaths/Recovered are raw counts and will almost always look strongly correlated with each other simply because bigger countries have bigger numbers across the board (scale effect). The **rate** columns strip scale out of the picture — so if Death Rate and Recovery Rate still show a meaningful correlation here, that's a genuine relationship about outcomes, not just both columns being driven by country size.

Outlier Detection (IQR Rule)

Earlier, boxplots showed outliers visually. This step identifies them explicitly by name using the standard IQR rule: any value more than $1.5 \times \text{IQR}$ beyond the 25th/75th percentile is flagged as an outlier.

```
In [77]: Q1 = latest_df["Confirmed"].quantile(0.25)
Q3 = latest_df["Confirmed"].quantile(0.75)
IQR = Q3 - Q1

outliers = latest_df[
    (latest_df["Confirmed"] < (Q1 - 1.5*IQR)) |
    (latest_df["Confirmed"] > (Q3 + 1.5*IQR))
]
```

```
print(outliers[["Country", "Confirmed"]].sort_values("Confirmed", ascending=False))
```

	Country	Confirmed
49030	US	4290259
48835	Brazil	2442375
48936	India	1480073
48992	Russia	816680
49005	South Africa	452529
48963	Mexico	395489
48986	Peru	389717
48854	Chile	347923
49028	United Kingdom	300111
48938	Iran	293606
48982	Pakistan	274289
49006	Spain	272421
48997	Saudi Arabia	268934
48888	Colombia	257101
48942	Italy	246286
49018	Turkey	227019
48827	Bangladesh	226225
48921	France	208665
48925	Germany	207112
48813	Argentina	167416
48939	Iraq	112585
48990	Qatar	109597
48937	Indonesia	100303
48903	Egypt	92482
48946	Kazakhstan	84648
48987	Philippines	82040
48902	Ecuador	81161
49010	Sweden	79395
48981	Oman	77058
48833	Bolivia	71181
48868	China	68135
48829	Belarus	67251
49020	Ukraine	67096
48830	Belgium	66428
48949	Kuwait	64379
48901	Dominican Republic	64156
48941	Israel	63985
48983	Panama	61442
49021	United Arab Emirates	59177
48850	Canada	58728
48974	Netherlands	53151
49001	Singapore	50838
48989	Portugal	50299
48991	Romania	45902
48928	Guatemala	45309

Interpretation: This lists every country whose confirmed case count is unusually high compared to the overall distribution. In a right-skewed dataset like this one, expect this list to be one-sided — i.e., outliers on the high end (major outbreak countries), not the low end, since case counts can't go below zero.

Mortality Rate vs Case Fatality Rate

These two sound similar but measure different things:

- **Case Fatality Rate (CFR)** = $\text{deaths} \div \text{confirmed cases}$. "Of the people we know got infected, what fraction died?"
- **Mortality Rate** = $\text{deaths} \div \text{total population}$. "Out of everyone in the world, what fraction died from this?"

CFR will always look much higher than mortality rate, because the denominator is far smaller (confirmed cases, not the whole population). This comparison is a good way to show you understand the difference between the two — a common point of confusion in epidemiology.

```
In [78]: total_deaths = latest_df["Deaths"].sum()
total_confirmed = latest_df["Confirmed"].sum()
total_population = 7_800_000_000 # approximate world population at the time of th

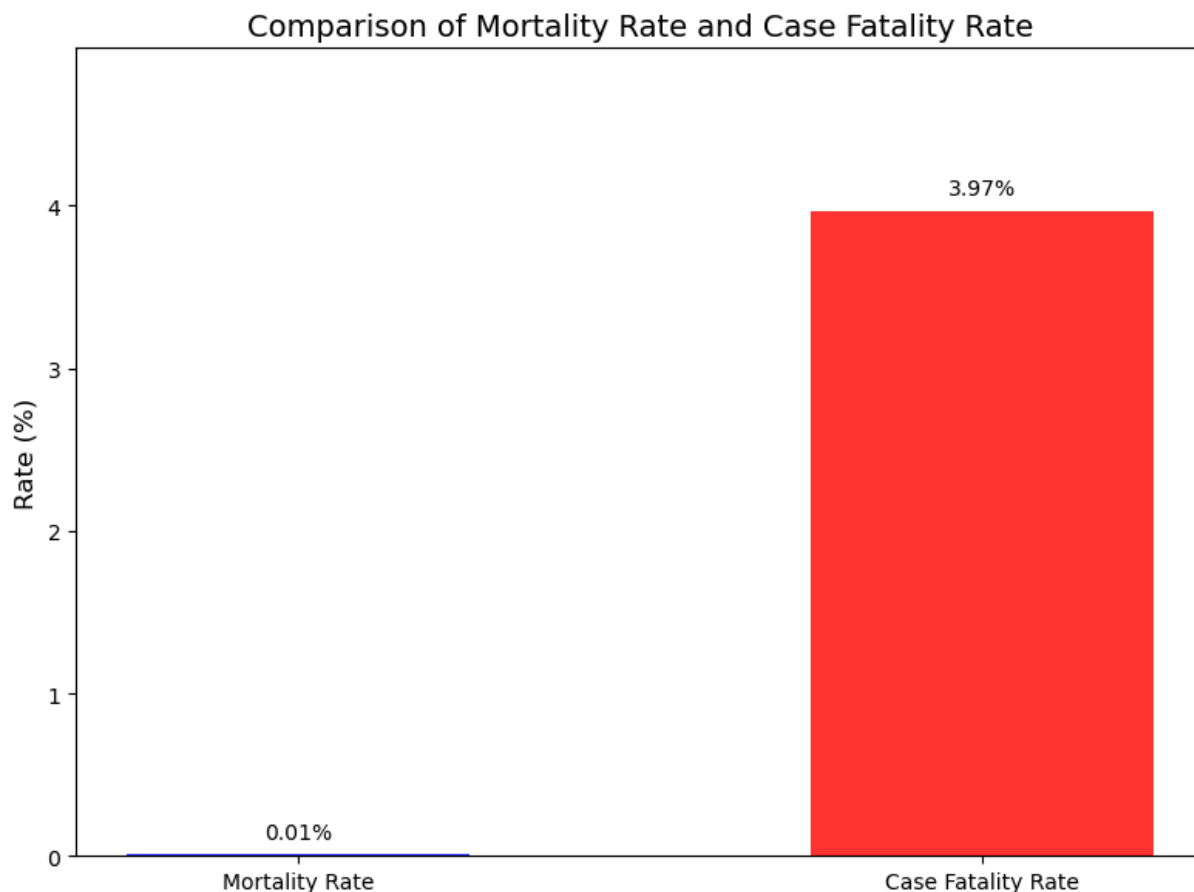
mortality_rate = (total_deaths / total_population) * 100
case_fatality_rate = (total_deaths / total_confirmed) * 100

fig, ax = plt.subplots(figsize=(8, 6))
x_labels = ['Mortality Rate', 'Case Fatality Rate']
y_values = [mortality_rate, case_fatality_rate]

ax.bar(x_labels, y_values, color=['blue', 'red'], alpha=0.8, width=0.5)
ax.set_title('Comparison of Mortality Rate and Case Fatality Rate', fontsize=14)
ax.set_ylabel('Rate (%)', fontsize=12)
ax.set_ylim(0, max(y_values) + 1)

for i, v in enumerate(y_values):
    ax.text(i, v + 0.1, f'{v:.2f}%', ha='center', fontsize=10)

plt.tight_layout()
plt.show()
```



Note: `total_population` is a fixed estimate of world population (~7.8 billion), not something pulled from this dataset — this dataset only tracks cases, not population. This is a reasonable approximation for illustration, but it's worth stating clearly in your report that it's an external constant, not derived data, if your teacher asks where the number came from.

Summary

Key Findings

- COVID-19 confirmed case counts are **highly right-skewed** — a small number of countries (US, India, Brazil) account for a disproportionate share of global cases, confirmed by both the positive skewness value and the outlier list above.
- **Confirmed cases and deaths show a strong positive correlation** in raw counts, but this is partly a scale effect — bigger countries have bigger numbers across every column.
- **Recovery and death rates vary considerably** between countries, which likely reflects differences in healthcare capacity, testing thoroughness, and how each country reported its data — not necessarily the severity of the outbreak itself.
- Several countries appear as **statistical outliers** by the IQR rule due to exceptionally high case counts, consistent with what the boxplots showed visually.

- Given the high variability (high CV, high std relative to mean), **median and IQR are more representative than mean** for describing the "typical" country in this dataset — the mean is pulled upward by a handful of extreme outbreaks.
- **Case Fatality Rate is much higher than Mortality Rate**, which is expected since CFR is calculated only against confirmed cases, while mortality rate is calculated against the entire world population.